

Trabajo Práctico 7

I. Sistema de tipos

1. ¿Qué es un sistema de tipos y cuál es su principal función?
2. Definir y contrastar las definiciones de un sistema de tipos fuerte y débil (probablemente en la bibliografía se encuentren dos definiciones posibles. Volcar ambas en la respuesta). Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.
3. Además de la clasificación anterior, también es posible caracterizar el tipado como estático o dinámico. ¿Qué significa esto? Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar

1. ¿Qué es un sistema de tipos y cuál es su principal función?

Un **tipo de datos** se define como un **conjunto de valores y un conjunto de operaciones** definidas sobre dichos valores. Por extensión, un **sistema de tipos** (enmarcado dentro de la semántica del lenguaje) es el mecanismo encargado de **comprobar el correcto sentido de las sentencias** construidas.

Su **función principal** es garantizar que las operaciones se realicen sobre valores válidos y compatibles, detectando errores semánticos (como intentar sumar tipos incompatibles) para asegurar la integridad de los datos y el comportamiento esperado del programa.

2. Tipado fuerte y débil

Aunque las definiciones exactas de "fuerte" y "débil" no aparecen con esos nombres en los fragmentos de texto, los conceptos se describen a través de la **flexibilidad de las reglas de compatibilidad** y el manejo de **efectos colaterales**. (Nota: Las definiciones a continuación son conceptos estándar en la bibliografía de lenguajes de programación y se ejemplifican con los casos citados en las fuentes).

- **Definición 1 (Conversiones implícitas):** Se centra en si el lenguaje permite o no mezclar tipos en operaciones sin una conversión explícita.
- **Definición 2 (Seguridad de tipo):** Se refiere a la capacidad del lenguaje para asegurar que una variable solo contenga valores del tipo declarado y prohíba interpretaciones erróneas de la memoria.

Ejemplos y justificación:

- **Tipado Débil (Ejemplo: C):** Se considera débil porque permite **sobrecargar operadores** y realizar operaciones entre tipos diferentes, como sumar un entero y un real directamente. Además, representa valores booleanos mediante enteros (donde 0 es falso y cualquier otro valor es verdadero), permitiendo mezclar lógica y aritmética.
- **Tipado Fuerte (Ejemplo: Pascal / Ada):** En **Pascal**, el compilador asegura mediante tipos de **subrango** que un valor no exceda los límites definidos (ej. un mes entre 1 y 12), evitando errores de lógica. En **Ada**, se utiliza el **encapsulamiento** para que la manipulación de los datos solo pueda realizarse a través de las operaciones abstractas definidas, impidiendo accesos ilegales a la estructura interna.

3. Tipado estático vs. dinámico

Esta clasificación se refiere al momento en que se realizan las **ligaduras** de tipo y las comprobaciones.

- **Tipado Estático:** Las ligaduras de tipo se determinan y verifican durante la **fase de compilación**. El compilador analiza el código fuente para asegurar que las variables y subprogramas se usen de acuerdo a su **signatura**.
 - **Ejemplo 1: Java.** Es estático porque el tipo de un subprograma se comprueba cuando se realiza la llamada para detectar errores semánticos antes de la ejecución.
 - **Ejemplo 2: Pascal.** El compilador verifica las declaraciones de tipos y subrangos, asegurando que las funciones no sean invocadas con valores fuera de los límites especificados.
- **Tipado Dinámico:** Las ligaduras correspondientes se llevan a cabo durante la **ejecución** y no en la compilación. El tipo está asociado al valor (r-valor) y no al nombre de la variable.
 - **Ejemplo 1: Ruby.** Las fuentes lo categorizan explícitamente como un lenguaje con tipado dinámico.
 - **Ejemplo 2: PHP.** Al igual que Ruby, se identifica como un lenguaje donde las listas y otras estructuras son tipos dinámicos propios del lenguaje.

II. Tipos de datos

1. Dar una definición de tipo de dato.
2. ¿Qué es un tipo predefinido elemental? Dar ejemplos.
3. ¿Qué es un tipo definido por el usuario? Dar ejemplo

Definición de tipo de dato

Un **tipo de datos** se define fundamentalmente como un **conjunto de valores y un conjunto de operaciones** que se pueden utilizar para manipular dichos valores. Esta especificación permite proteger a las variables de operaciones no permitidas y ayuda al compilador o intérprete a detectar errores de forma temprana, aportando confiabilidad al código.

Tipos predefinidos elementales

Los **tipos predefinidos elementales** (también llamados primitivos, simples o *built in*) son aquellos que **vienen descritos en la definición del lenguaje** y no pueden descomponerse en otros tipos más básicos. Actúan como **abstracciones del hardware** subyacente y sus valores se ligán a una representación de máquina según la arquitectura del sistema.

Sus principales ventajas incluyen la **verificación estática**, el control de precisión y la invisibilidad de su representación física para el programador. Algunos ejemplos son:

- **Enteros:** Representan un subconjunto de los números enteros matemáticos.
- **Reales:** Utilizados para números decimales en notación de punto flotante o fijo.
- **Caracteres:** Permiten el intercambio de información mediante tablas como ASCII o Unicode.
- **Booleanos:** Valores lógicos que consisten únicamente en "verdadero" o "falso".

Tipos definidos por el usuario

Son aquellos **creados por el programador** mediante el uso de **constructores de tipos**. Estos permiten definir nuevos tipos a partir de los tipos base o predefinidos, facilitando la creación de **abstracciones**, el encapsulamiento de lógica y la mejora en la claridad del código

Ejemplos de estos tipos incluyen:

- **Registros (Structs):** Permiten agrupar elementos de distinto tipo (heterogéneos) bajo un mismo nombre. Por ejemplo, un registro `Fraccion` que contenga dos enteros: `numerador` y `denominador`.
- **Arreglos (Arrays):** Estructuras de tamaño fijo donde todos los elementos son del mismo tipo. Por ejemplo, un arreglo para almacenar una lista de 10 enteros.
- **Enumerados:** Definen un número reducido y ordenado de valores con nombre, como un tipo `DiaSemana` que contenga `(Lunes, Martes, Miercoles...)`.
- **Subrangos:** Creados especificando un intervalo de otro tipo simple, como un tipo `Mes` definido entre los enteros `1..12`.

III. Tipos compuestos

1. Dar una breve definición de: producto cartesiano (en la bibliografía puede aparecer también como product type), correspondencia finita, uniones (en la bibliografía puede aparecer también como sum type) y tipos recursivos.
2. Identificar a qué clase de tipo de datos pertenecen los siguientes extractos de código.

En algunos casos puede corresponder más de una:

Java <pre>class Persona { String nombre; String apellido; int edad; }</pre>	C <pre>typedef struct _nodoLista { void *dato; struct _nodoLista *siguiente } nodoLista; typedef struct _lista { int cantidad; nodoLista *primero } Lista;</pre>	C <pre>union codigo { int numero; char id; };</pre>
Ruby <pre>hash = { uno: 1, dos: 2, tres: 3, cuatro: 4 }</pre>	PHP <pre>function doble(\$x) { return 2 * \$x; }</pre>	Python <pre>tuple = ('physics', 'chemistry', 1997, 2000)</pre>
Haskell <pre>data ArbolBinarioInt = Nil Nodo int (ArbolBinarioInt dato) (ArbolBinarioInt dato)</pre> Ayuda para interpretar: 'ArbolBinarioInt' es un tipo de dato que puede ser Nil ("vacío") o un Nodo con un dato número entero (int) junto a un árbol como hijo izquierdo y otro árbol como hijo derecho	Haskell <pre>data Color = Rojo Verde Azul</pre> Ayuda para interpretar: 'Color' es un tipo de dato que puede ser Rojo, Verde o Azul.	

Definiciones

- **Producto Cartesiano (Registros/Structs/Product Type):** Este constructor define un conjunto de n-tuplas ordenadas. Su función es **agrupar campos que pueden ser de diferentes tipos** (heterogéneos) bajo un único nombre identificador,. El acceso a estos componentes se realiza habitualmente mediante la **notación punto** (ej. `objeto.campo`),.
- **Correspondencia Finita (Arreglos/Arrays):** Se define como una función que mapea un dominio de tipo índice (generalmente un subrango de números enteros) a un rango de valores de un tipo de dato determinado. Son estructuras de **tamaño fijo y componentes homogéneos** (todos los elementos deben ser del mismo tipo),.
- **Uniones (Sum Type / Unión Discriminada):** Es un mecanismo que permite que una misma ubicación de memoria almacene **diferentes tipos de datos en distintos momentos** de la ejecución, representando una disyunción de tipos. Las **uniones discriminadas** son una variante más

segura que incluye un "tag" o discriminante para indicar qué tipo de dato está activo en ese momento, facilitando el chequeo de tipos.

- **Tipos Recursivos:** Un tipo recursivo se define como una estructura que puede contener **componentes de su mismo tipo**. Esta característica es fundamental para la implementación de estructuras de datos complejas como listas ligadas, árboles y grafos.

Casos de Extractos de código

Caso Java

```
class Persona {  
    String nombre;  
    String apellido;  
    int edad;  
}
```

En este caso corresponde a un tipo de producto cartesiano, debido a que este cubre las características del mismo. Incluyendo la notación de acceso, y su conjunto de n-tuplas ordenadas.

Casos de C

```
typedef struct _nodoLista {  
    void *dato;  
    struct _nodoLista *siguiente;  
} nodoLista;  
  
typedef struct _lista {  
    int cantidad;  
    nodoLista *primero;  
} Lista;
```

- **Producto Cartesiano (Registros/Structs):** Se identifica por el uso de `struct`. Siendo un conjunto de n-tuplas que permite **agrupar campos de diferentes tipos** (como `int` y punteros) bajo un mismo nombre.
- **Recursión (Tipos Recursivos):** La estructura `_nodoLista` es un tipo recursivo porque **contiene un componente de su mismo tipo** (el puntero `struct _nodoLista *siguiente`). Es la base para implementar estructuras como las **listas ligadas**.

```
union codigo {  
    int numero;  
    char id;  
};
```

Unión: El código utiliza la palabra reservada `union`. Este mecanismo permite que una **misma ubicación de memoria** almacene diferentes tipos de datos (`int` o `char`) en distintos momentos de la ejecución. No contiene ningún mecanismo interno para saber qué tipo de dato está almacenado en un momento dado.

Caso Ruby

```
hash = {  
    uno: 1,  
    dos: 2,  
    tres: 3,  
    cuatro: 4  
}
```

Un `hash` en Ruby (o diccionario en otros lenguajes) funciona como una **función o mapeo** que relaciona un dominio de etiquetas (las claves como `:uno`, `:dos`) con un rango de valores (los números 1, 2, etc.).

En síntesis, el constructor utilizado es correspondencia finita. En una correspondencia finita como el `hash`, el acceso es mediante una **clave** que actúa como un índice dentro de un dominio.

Caso Php

```
function doble($x) {  
    return 2 * $x;  
}
```

- **Rutina (Función):** Este extracto representa una **acción abstracta** encapsulada. Al devolver un valor explícito mediante `return`, se clasifica específicamente como una **función**.
- **Tipo por Signatura:** El "tipo" de esta unidad de programa no es un dato elemental, sino que viene dado por su **signatura** (sus parámetros y su valor de retorno).

Caso Python

```
tuple = ('physics', 'chemistry', 1997, 2000)
```

Este constructor es producto cartesiano, un "**conjunto de n-tuplas ordenadas**". Su característica principal es que permite agrupar componentes que pueden ser de **diferentes tipos** (heterogéneos). En este ejemplo la tupla contiene tanto cadenas de texto ('physics' , 'chemistry') como números enteros (1997 , 2000), lo cual encaja perfectamente con la definición de un registro o "product type" que maneja datos de tipo distinto bajo una estructura fija.

Casos Haskell

```
data ArbolBinarioInt =  
    Nil |  
    Nodo int  
        (ArbolBinarioInt dato)  
        (ArbolBinarioInt dato)  
  
-- Ayuda para interpretar:  
-- 'ArbolBinarioInt' es un tipo de dato que puede ser  
-- Nil ("vacío") o un Nodo con un dato número entero (int)  
-- junto a un árbol como hijo izquierdo y otro árbol como  
-- hijo derecho.
```

- **Unión (Sum Type / Disyunción):** El símbolo `|` indica que el tipo `ArbolBinarioInt` puede tomar una de dos formas alternativas: o es un `Nil` o es un `Nodo`. Esto representa una disyunción de tipos donde la memoria se interpreta de forma distinta según el caso activo.
- **Producto Cartesiano (Product Type):** La variante `Nodo` actúa como un **registro o estructura**, ya que agrupa tres componentes que coexisten simultáneamente: un valor de tipo `int` y dos referencias a otros árboles.,
- **Recursión:** Es el aspecto más crítico del código. Se trata de un **tipo recursivo**, definido como una estructura que contiene componentes de **su mismo tipo** (`ArbolBinarioInt`). La recursión es la base necesaria para implementar estructuras de datos de tamaño arbitrario como los árboles.
-

```
data Color =  
  Rojo |  
  Verde |  
  Azul  
  
-- Ayuda para interpretar:  
-- 'Color' es un tipo de dato que puede ser Rojo, Verde o  
Azul.
```

Está utilizando el constructor de **Unión** (específicamente una **Unión Discriminada** o *Sum Type*). Esto es así porque el símbolo `|` define una **disyunción de tipos**, indicando que una variable de tipo `Color` puede ser una alternativa u otra (Rojo, Verde o Azul), pero no todas a la vez. Permite un **chequeo de tipos seguro**.

IV. Mutabilidad/Inmutabilidad

1. Definir mutabilidad e inmutabilidad respecto a un dato. Dar ejemplos en al menos 2 lenguajes. TIP: indagar sobre los tipos de datos que ofrece Python y sobre la operación `#FREEZE` en los objetos de Ruby.
2. Dado el siguiente código:

```
a = Dato.new(1)  
a = Dato.new(2)
```

¿Se puede afirmar entonces que el objeto “Dato.new(1)” es mutable? Justificar la respuesta sea por afirmativa o por la negativa.

Mutabilidad e inmutabilidad

En los lenguajes de programación, estos conceptos se refieren a la capacidad (o incapacidad) de modificar el **R-Valor** (el contenido o valor almacenado) de una entidad en su **L-Valor** (ubicación de memoria) original tras haber sido creado.

- **Mutabilidad:** Se produce cuando un objeto permite que su estado o sus componentes internos sean modificados sin cambiar su identidad (su dirección de memoria). En este caso, las operaciones sobre el dato producen un **efecto colateral** que altera la celda de memoria original.

- **Inmutabilidad:** Un dato es inmutable cuando su valor no puede ser modificado una vez establecido. Cualquier intento de "cambio" resulta en la creación de un nuevo objeto en una ubicación de memoria distinta, dejando el original intacto (o disponible para el *Garbage Collector*).

Ejemplos en lenguajes de programación:

- **Python:** Python ofrece tipos **inmutables** como las cadenas de texto (*strings*), tuplas y números; si intentas modificar un carácter de un string, Python crea uno nuevo. Por otro lado, ofrece tipos **mutables** como las **listas**, que permiten insertar o eliminar elementos modificando la estructura original en la memoria.
- **Ruby:** En Ruby, la mayoría de los objetos son mutables por defecto. Sin embargo, el lenguaje ofrece la operación **#FREEZE**, que convierte a un objeto en inmutable; una vez que un objeto ha sido "congelado", cualquier intento de modificar sus atributos lanzará un error en tiempo de ejecución.
- **Pascal / C:** Las **constantes** son el ejemplo básico de inmutabilidad; una vez que se liga un nombre a un valor (ya sea en compilación o ejecución), el lenguaje prohíbe su modificación posterior.

Caso del código

```
a = Dato.new(1)
a = Dato.new(2)
```

No, no se puede afirmar que el objeto sea mutable basándose únicamente en ese código.

El código proporcionado muestra una **reasignación de variable**, no una **mutación de objeto**. Para entender esto, debemos observar la semántica de las ligaduras (bindings):

1. **Primera instrucción** (`a = Dato.new(1)`): Se crea una instancia de la unidad en la memoria **heap**. La variable `a` (el nombre) se liga a la dirección de memoria (L-valor) de ese nuevo objeto que contiene el valor `1`.
2. **Segunda instrucción** (`**a = Dato.new(2)**`): Se crea un **segundo objeto** distinto en la heap con el valor `2`. La variable `a` rompe su ligadura anterior y se liga a la dirección de este nuevo objeto.

En este proceso, el objeto original (`Dato.new(1)`) permanece inalterado en su propia celda de memoria hasta que el sistema lo considere "basura" por no tener más referencias. La **mutabilidad** requeriría una operación que cambiara el `1` por un `2` **dentro del mismo objeto** (por ejemplo, `a.valor = 2`), sin crear una nueva instancia con `new` .

Por lo tanto, lo que el código demuestra es que la **variable a tiene una ligadura dinámica** respecto a su valor, pero no dice nada sobre la capacidad del objeto `Dato` para ser modificado internamente.

V. Manejo de punteros

1. ¿Permite C tomar el l-valor de las variables? Ejemplificar.
2. ¿Qué problemas existen en el manejo de punteros? Ejemplificar.

L-valor variables en C

Sí, el lenguaje C permite tomar el **l-valor** (l-value) de las variables a través del uso de **punteros**. En la semántica de los lenguajes de programación, el l-valor se define como la **dirección del área de memoria** ligada a una variable durante la ejecución del programa.

Un puntero es una variable cuyo **r-valor** (el valor codificado almacenado en su celda) es la dirección de memoria o **l-valor de otra variable**.. Esto permite acceder a variables de forma indirecta utilizando su ubicación física en lugar de su nombre..

- **Ejemplo:** En C, si declaramos una variable entera y un puntero hacia ella, el r-valor del puntero será el l-valor (dirección) de la variable entera.. Esto constituye un camino de acceso (access path) directo al objeto original en la memoria..

Problemas de punteros

El uso de punteros es una herramienta potente, pero las fuentes indican que es "peligroso" y lo comparan con el *go to* de los datos debido a las siguientes inseguridades:

1. **Referencias sueltas (Dangling references):** Ocurre cuando un puntero apunta a una dirección de memoria que ya ha sido liberada..
2. **Objetos perdidos (Memory Leaks):** Se produce cuando se aloca memoria en el **heap** que luego deja de ser accesible por el programa, pero no ha sido

liberada explícitamente..

3. **Alias:** Es el riesgo de tener **múltiples nombres o punteros accediendo a la misma memoria**; cualquier modificación realizada a través de un alias afecta a todos los demás caminos de acceso, dificultando el rastreo de cambios.
4. **Violación de tipos:** Sucede al acceder a una posición de memoria e interpretarla con un tipo de dato incorrecto.
5. **Punteros no inicializados:** Intentar acceder a la memoria a través de un puntero que aún contiene "datos basura".
6. **Ejemplo de Alias:** Si dos punteros comparten un mismo objeto en un entorno de referencia, el objeto modificado por uno de los punteros aparecerá modificado para el otro, lo que puede generar efectos colaterales inesperados y programas difíciles de depurar.

VI. TAD

1. ¿Qué características debe cumplir una unidad para que sea un TAD?
2. Dar algunos ejemplos de TAD en lenguajes tales como ADA, Java, Python, entre otros.

Características

Un TAD debe reunir los siguientes requisitos:

- **Combinación de Datos y Operaciones:** Un TAD no es solo una estructura de datos; es la unión de un **conjunto de valores** y un **conjunto de operaciones abstractas** definidas específicamente para manipular esos valores.
- **Encapsulamiento:** Debe agrupar los datos y sus operaciones en una misma unidad sintáctica, de modo que la manipulación de la información solo pueda realizarse a través de las operaciones permitidas.
- **Ocultamiento de Información (Information Hiding):** Debe distinguir entre una interfaz **pública** (las operaciones que el usuario puede ver y usar) y una implementación **privada** (los detalles internos de cómo se guardan los datos, los cuales son invisibles para el usuario).
- **Abstracción:** Permite al programador ignorar los detalles de implementación y pensar en términos de "qué hace" la unidad en lugar de "cómo lo hace".

- **Identificación Unívoca:** Debe tener un **nombre** asignado que lo identifique y las operaciones deben estar descritas como un conjunto de rutinas especificadas en su declaración.

Ejemplos TAD

El soporte para implementar TADs varía según el lenguaje y sus mecanismos de abstracción:

- **Ada:** Utiliza el concepto de **paquete (package)** para encapsular las definiciones de tipos y las operaciones permitidas, ocultando la representación interna del usuario.
- **Java:** Implementa TADs principalmente mediante **clases**, que representan la evolución final del TAD en el paradigma orientado a objetos. Ejemplos comunes en sus librerías son las interfaces como **List**, que ocultan si la implementación subyacente es un **ArrayList** (basado en arreglos) o un **LinkedList** (basado en nodos vinculados).
- **Python:** Aunque es un lenguaje con esquema dinámico, utiliza **clases** para manejar la abstracción de datos y permite implementar estructuras como **pilas, colas y árboles**.
- **Ejemplos universales:** Independientemente del lenguaje, los TADs más frecuentes incluyen **pilas, colas, listas ligadas, árboles y grafos**.